

부록 C

전체 코드

본문에는 핵심 몇 줄만 실었습니다. 여기에는 저자 PC에서 실제로 돌려본 코드 전문을 모았습니다. 직접 칠 필요는 없습니다. AI에게 "부록 C의 ○○.py를 이 폴더에 만들어줘"라고 하거나, 파일째 옮겨 쓰시면 됩니다.

파이썬 설치와 필요한 부품은 부록 A에 있습니다. 의미 검색(6-4)만 추가 부품이 크니 그 장에서만 설치하세요.

파일	쓰이는 장	하는 일
gen_단가보고.py	4-5	월간 단가 변동 보고서를 다시 만드는 재생성 스크립트
연습폴더만들기.py	6부 공통	연습용 가상 사무 문서 100개를 만든다
폴더실측.py	6-1	폴더 안 글이 AI 한 번에 다 들어가는지 쟀다
키워드검색.py	6-2	폴더의 .md/.txt를 색인하고 단어로 찾는다
토큰나이저비교.py	6-2-6-3	한국어에서 색인 방식별로 무엇이 찾히는지 비교한다
의미검색.py	6-4	뜻이 가까운 문서를 찾는다(내 PC에서 도는 작은 모델)
섞어찾기.py	6-5	키워드 검색과 의미 검색 순위를 섞는다(RRF)
채점_12문항.py	6-5	정답을 아는 질문 12개로 세 방식을 채점한다
check.ps1	9-1	아침 자동 실행 결과를 세 질문으로 점검한다

gen_단가보고.py (4-5)

월간 단가 변동 보고서를 다시 만드는 재생성 스크립트

```

# gen_단가보고.py - 월간 단가 변동 보고서를 다시 만드는 절차
# 사용법: python gen_단가보고.py 2026-10
# → 같은 폴더의 '단가_2026-10.xlsx'를 읽어 '단가보고_2026-10.docx'를 만든다
import os
import sys
import openpyxl
from docx import Document
from docx.shared import Pt
from docx.oxml.ns import qn

# —— 1. 이번 달 설정 (매달 바뀌는 건 이것뿐) ——
MONTH = sys.argv[1] if len(sys.argv) > 1 else "2026-09"
SRC = f"단가_{MONTH}.xlsx"
OUT = f"단가보고_{MONTH}.docx"
FONT = "맑은 고딕"
if not os.path.exists(SRC):
    sys.exit(f"[중단] 원본 파일이 없습니다: {SRC}")

# —— 2. 원본 읽기 (엑셀 첫 줄은 제목 줄) ——
ws = openpyxl.load_workbook(SRC, data_only=True).active
rows = [r for r in ws.iter_rows(min_row=2, values_only=True) if r[0]]

# —— 3. 계산 ——

items = []
for name, vendor, old, new, qty in rows:
    items.append((name, vendor, old, new, (new - old) * qty))
total = sum(i[4] for i in items)
up = [i for i in items if i[4] > 0]

# —— 4. 문서 만들기 ——

doc = Document()
st = doc.styles["Normal"]
st.font.name, st.font.size = FONT, Pt(10)
st.element.rPr.rFonts.set(qn("w:eastAsia"), FONT)

```

```

doc.add_heading(f"{MONTH} 거래처 단가 변동 보고", level=1)
doc.add_paragraph(
    f"결론: 이번 달 단가 변동으로 월 비용이 {abs(total):,}원 "
    f"'늘어납니다' if total > 0 else '줄어듭니다'}. "
    f"인상 품목 {len(up)}건 중 영향이 가장 큰 것은 "
    f"'{max(items, key=lambda i: i[4])[0]}'입니다.")

t = doc.add_table(rows=1, cols=5)
t.style = "Table Grid"
for c, h in zip(t.rows[0].cells, ["품목", "거래처", "이전 단가", "신규 단가", "월
    영향(원)"]):
    c.text = h
for name, vendor, old, new, diff in items:
    cells = t.add_row().cells
    for c, v in zip(cells, [name, vendor, f"{old:,}", f"{new:,}", f"{diff:+,}"]):
        c.text = v

doc.save(OUT)
print(f"[생성 완료] {OUT} (품목 {len(rows)}건, 합계 {total:+,}원)")

# —— 5. 검사: 한 달에 30% 넘게 바뀐 단가는 입력 실수일 수 있다 ——
for name, vendor, old, new, diff in items:
    if abs(new - old) > old * 0.3:
        print(f"[확인필요] {name}: {old:,}원 → {new:,}원 (30% 넘게 변동)")

```

연습폴더만들기.py (6부 공통)

연습용 가상 사무 문서 100개를 만든다

```

# [부록 C] 『내 문서가 달라졌다』 6부 공통 - 저자 PC에서 2026-09-28 실행 확인
(Python 3.12, SQLite 3.49, CPU)
# 연습용 샘플 폴더 100개 파일 생성 (가상의 사무 문서)
import os, random
random.seed(7)
ROOT = os.path.join(os.path.dirname(__file__), "샘플폴더")
os.makedirs(ROOT, exist_ok=True)

거래처 = ["A상사", "B물산", "C상사", "D정밀", "E유통", "F산업"]
품목 = ["브래킷", "커버 패널", "고무 패킹", "배선 묶음", "받침대", "나사 세트"]
사람 = ["김 과장", "이 대리", "박 차장", "최 주임", "정 팀장"]

filler = [
    "{c}에서 {p} 견적서를 다시 보내 달라는 연락이 왔습니다.",
    "{p} 수량은 지난달과 같게 유지하기로 했습니다.",
    "{n}이 다음 주까지 발주서 초안을 정리하기로 했습니다.",
    "월말 마감 전에 {c} 거래 내역을 한 번 더 대조합니다.",
    "{p} 도면 개정본은 공유 폴더에 올려두었습니다.",
    "회의는 30분 안에 끝났고 특이사항은 없었습니다.",
    "{c} 담당자가 바뀌어서 연락처를 새로 받았습니다.",
    "{p} 입고 검수는 평소대로 진행했습니다.",
    "다음 분기 계획은 {n}이 초안을 잡기로 했습니다.",
    "{c}와의 통화 내용을 아래에 요약합니다.",
    "납기는 계약서에 적힌 날짜를 기준으로 관리합니다.",
    "{p} 납기 일정표를 최신으로 고쳤습니다.",
    "견적서 양식의 부가세 칸을 별도로 분리했습니다.",
    "{n}이 {c} 방문 일정을 잡았습니다.",
    "품질 관련 문의는 이번 달 두 건이었습니다.",
    "보고서 표의 합계를 원본 데이터와 대조했습니다.",
]

kinds = ["회의록", "견적서", "발주서", "월보고", "통화메모", "업무일지"]

def filler_doc(k, i):
    lines = []
    for _ in range(random.randint(12, 22)):
        s = random.choice(filler).format(c=random.choice(거래처),
        p=random.choice(품목), n=random.choice(사람))
        lines.append(s)

```

```
body = []
for j in range(0, len(lines), 3):
    body.append(" ".join(lines[j:j+3]))
return f"# {k} {i:03d}\n\n" + "\n\n".join(body) + "\n"
```

정답이 정해진 문서 12개 (검색 시험용)

```
planted = {
    "회의록_2025-11-03.md": "# 주간 회의록 2025-11-03\n\nB물산 입고가 예정보다 2주  
늦어졌습니다. 우리 쪽 생산 일정도 같이 밀려서 고객에게 사과 공문을 보내기로  
했습니다.\n\n늦어진 이유는 B물산 협력사의 급형 수리 때문이라고 합니다. 다음  
입고일은 11월 17일로 다시 받았습니다.\n",
    "클레임_2025-09-12.md": "# 클레임 처리 기록\n\nD정밀에서 받은 박스가 찢어진  
채로 들어왔고, 안에 든 부품 두 개가 깨져 있었습니다. 사진을 찍어 보내고  
교환을 요청했습니다.\n\nD정밀은 운송 중 충격으로 보고 완충재를 두 겹으로  
바꾸겠다고 답했습니다.\n",
    "통화메모_2025-10-02.md": "# 통화 메모 - E유통\n\n원자재 값이 올라서 다음 달부터  
개당 가격을 7% 올려 받겠다는 통보가 왔습니다. 연간 계약 물량이면 4%까지 조정  
가능하다고 합니다.\n\n팀장님 보고 후 답을 주기로 했습니다.\n",
    "업무일지_2025-08-21.md": "# 업무일지 8월 21일\n\nC상사가 석 달째 돈을 안 보내고  
있습니다. 세금계산서는 6월에 발행했고, 담당자는 내부 결재가 늦어진다는 말만  
반복합니다.\n\n내용증명을 보내기 전에 한 번 더 방문하기로 했습니다.\n",
    "출장보고_2025-07-15.md": "# 출장 보고\n\n출장 경비 정산 내역을 첨부합니다.  
교통비와 숙박비는 영수증 기준이고 식대는 규정 한도 안입니다.\n\n다음  
출장부터는 법인카드로 결제하기로 했습니다.\n",
    "부품목록_2025.md": "# 부품 목록 2025\n\n품번 KX-2041 은 받침대 하단 고정용  
부품입니다. 대체 품번은 없고 최소 주문 수량은 500개입니다.\n\n품번 KX-2040 은  
단종되었습니다.\n",
    "인사메모_2025-06-30.md": "# 인사 관련 메모\n\n지난달 밤 늦게까지 남아서 일한  
시간에 대한 추가 급여가 이번 달 급여에 반영됩니다. 신청서는 팀장 확인 후  
총무팀으로 보냅니다.\n",
    "창고현황_2025-10-20.md": "# 창고 현황\n\n창고에 남은 고무 패킹이 20개뿐이라  
이번 주 주문을 다 못 채웁니다. 추가 생산분은 다음 주 수요일에 들어옵니다.\n",
    "계약메모_2025-12-01.md": "# 계약 메모\n\nA상사와 재계약 조건을 정리했습니다.  
기간은 1년, 단가는 동결, 결제 조건은 60일에서 45일로 줄였습니다.\n",
    "품질기준_v2.md": "# 품질 검사 기준 v2\n\n입고품은 외관, 치수, 수량 세 가지를  
확인합니다. 치수는 도면 공차 안에 들어와야 하고, 외관 불량은 사진과 함께  
기록합니다.\n",
    "월보고_2025-03.md": "# 3월 월간 보고\n\n3월 매출은 전월보다 소폭 늘었습니다.  
신규 거래처 한 곳이 추가되었고 반품은 없었습니다.\n",
    "교육자료_신입.md": "# 신입 안내\n\n처음 오신 분은 공유 폴더 구조부터 익히세요.  
거래처별 폴더 안에 견적, 발주, 클레임이 나뉘어 있습니다. 모르는 건 옆자리에
```

```

        바로 물어보세요.\n",
    }

n = 0
for name, text in planted.items():
    open(os.path.join(ROOT, name), "w", encoding="utf-8").write(text); n += 1
i = 0
while n < 97:
    k = kinds[i % len(kinds)]
    ext = ".md" if i % 3 else ".txt"
    open(os.path.join(ROOT, f"{k}_{i:03d}{ext}"), "w", encoding="utf-8").write(filler_doc(k, i))
    i += 1; n += 1
import docx
d = docx.Document(); d.add_heading("발주 변경 통보", 1)
d.add_paragraph("F산업이 배선 묶음 납품 일정을 3주 뒤로 미루겠다고 통보했습니다. 사유는 원자재 입고 지연입니다.")
d.save(os.path.join(ROOT, "발주변경_2025-10.docx")); n += 1
open(os.path.join(ROOT, "단가표_2025.xlsx"), "wb").write(b"PK\x03\x04 dummy"); n += 1
open(os.path.join(ROOT, "회사소개.hwp"), "wb").write(b"HWP dummy"); n += 1
print("생성:", n, "개 →", ROOT)

```

폴더실측.py (6-1)

폴더 안 글이 AI 한 번에 다 들어가는지 찬다

```

# [부록 C] 『내 문서가 달라졌다』 6-1 - 저자 PC에서 2026-09-28 실행 확인 (Python
3.12, SQLite 3.49, CPU)
# 폴더실측.py - 폴더 안 글이 모두 몇 자인지, AI 한 번에 다 들어가는지 쟀다
# 사용: python 폴더실측.py "C:\공유폴더\영업"
import sys, pathlib

폴더 = pathlib.Path(sys.argv[1])
읽을확장자 = {".md", ".txt"}
글자수, 파일수, 못읽음 = 0, 0, {}
가장큰 = []
for p in 폴더.rglob("*"):
    if not p.is_file():
        continue
    if p.suffix.lower() in 읽을확장자:
        n = len(p.read_text(encoding="utf-8", errors="ignore"))
        글자수 += n
        파일수 += 1
        가장큰.append((n, p.name))
    else:
        못읽음[p.suffix.lower()] = 못읽음.get(p.suffix.lower(), 0) + 1

print(f"글 파일 {파일수}개, 합계 {글자수:,}자")
for n, 이름 in sorted(가장큰, reverse=True)[:3]:
    print(f" 가장 긴 파일: {이름} ({n:,}자)")
for 확장자, 개수 in 못읽음.items():
    print(f" 이 방법으로는 못 읽은 파일 {확장자}: {개수}개")

try:
    # 토큰 수(AI가 글을 세는 단위). tiktoken 이
    있으면 쏜다: pip install tiktoken
    import tiktoken
    enc = tiktoken.get_encoding("o200k_base")
    토큰 = sum(len(enc.encode(p.read_text(encoding="utf-8", errors="ignore"))))
    for p in 폴더.rglob("*") if p.is_file() and p.suffix.lower() in
    읽을확장자)
    print(f"토큰 약 {토큰:,}개 (글자 1자당 약 {토큰 / max(글자수, 1):.2f}토큰)")
except ImportError:
    print("토큰 수는 건너뛴 (pip install tiktoken 후 다시 실행)")

```

키워드검색.py (6-2)

폴더의 .md/.txt를 색인하고 단어로 찾는다

```

# [부록 C] 『내 문서가 달라졌다』 6-2 - 저자 PC에서 2026-09-28 실행 확인 (Python
3.12, SQLite 3.49, CPU)
# 키워드검색.py - 폴더의 .md/.txt 를 색인하고 단어로 찾는다 (파이썬 기본 기능만
사용)
# 사용: python 키워드검색.py 색인 "C:\공유폴더\영업"
#       python 키워드검색.py 찾기 납기 지연
import sqlite3, sys, pathlib

DB = pathlib.Path(__file__).with_name("검색색인.db") # 색인 파일(=찾아보기
목록)
읽을확장자 = {".md", ".txt"}

def 색인(폴더):
    con = sqlite3.connect(DB)
    con.execute("DROP TABLE IF EXISTS 문서")
    # trigram: 글자를 3개씩 겹쳐 잘라 둔다 → '견적서를' 안의 '견적서'도 찾는다
    con.execute("CREATE VIRTUAL TABLE 문서 USING fts5(경로, 본문,
tokenize='trigram')")
    넣음, 건너뛴 = 0, {}
    for p in pathlib.Path(폴더).rglob("*"):
        if not p.is_file():
            continue
        if p.suffix.lower() not in 읽을확장자:
            건너뛴[p.suffix.lower()] = 건너뛴.get(p.suffix.lower(), 0) + 1
            continue
        본문 = p.read_text(encoding="utf-8", errors="ignore")
        con.execute("INSERT INTO 문서 VALUES (?, ?)", (str(p), 본문))
        넣음 += 1
    con.commit()
    print(f"색인 완료: {넣음}개")
    for 확장자, 개수 in 건너뛴.items():
        print(f" 건너뛴 {확장자} or '(확장자 없음)': {개수}개") # ← 못 찾을
        때 제일 먼저 볼 곳

def 찾기(단어들, 몇개=5):
    con = sqlite3.connect(DB)
    # 두 글자 단어는 trigram 검색(MATCH)에 안 걸리므로 LIKE 로 찾는다
    조건 = " OR ".join("본문 LIKE ?" for _ in 단어들)
    행들 = con.execute(f"SELECT 경로, 본문 FROM 문서 WHERE {조건}",
[f"%{w}%" for w in 단어들]).fetchall()

```

```

결과 = []
for 경로, 본문 in 행들:
    맞은단어 = sum(1 for w in 단어들 if w in 본문) # 검색어가 몇 종류나 들어
    있나
    횟수 = sum(본문.count(w) for w in 단어들) # 몇 번 나오나
    결과.append((맞은단어, 횟수, 경로, 본문))
결과.sort(reverse=True)
for 맞은단어, 횟수, 경로, 본문 in 결과[:몇개]:
    i = 본문.find(next(w for w in 단어들 if w in 본문))
    print(f"[{맞은단어}종류/{횟수}회] {pathlib.Path(경로).name}")
    print("..." + 본문[max(0, i -
20):i + 40].replace("\n", " ") + "...")
    if not 결과:
        print("찾은 문서 없음 - '없다'가 아니라 '이 단어로는 못 찾았다'입니다")
    return [r[2] for r in 결과]

if __name__ == "__main__":
    if sys.argv[1] == "색인":
        색인(sys.argv[2])
    else:
        찾기(sys.argv[2:])

```

토크나이저비교.py (6-2-6-3)

한국어에서 색인 방식별로 무엇이 찾히는지 비교한다

```
# [부록 C] 『내 문서가 달라졌다』 6-2 · 6-3 - 저자 PC에서 2026-09-28 실행 확인
(Python 3.12, SQLite 3.49, CPU)

import sqlite3, pathlib
ROOT = pathlib.Path("샘플폴더")
docs = [(p.name, p.read_text(encoding="utf-8")) for p in sorted(ROOT.iterdir())
        if p.suffix in (".md", ".txt")]
con = sqlite3.connect(":memory:")
for tok in ["unicode61", "trigram"]:
    con.execute(f"CREATE VIRTUAL TABLE t_{tok} USING fts5(name, body,
        tokenize='{tok}')")
    con.executemany(f"INSERT INTO t_{tok} VALUES (?,?)", docs)
def n_match(tok, q):
    try: return len(con.execute(f"SELECT name FROM t_{tok} WHERE t_{tok}
        MATCH ?", (q,)).fetchall())
    except Exception as e: return f"오류:{e}"
def n_like(tok, w):
    return len(con.execute(f"SELECT name FROM t_{tok} WHERE body LIKE ?",
        (f"%{w}%",)).fetchall())
import re
for w in ["견적서", "납기", "계약", "단가", "재계약", "KX-2041", "늦어져서",
    "늦어"]:
    truth = sum(1 for _, b in docs if w in b)
    print(f"{w!r:12} 실제포함={truth:3} unicode61={n_match('unicode61',
        chr(34)+w+chr(34))!s:>4} trigram={n_match('trigram',
        chr(34)+w+chr(34))!s:>4} trigram=LIKE={n_like('trigram', w)}")
```

의미검색.py (6-4)

뜻이 가까운 문서를 찾는다(내 PC에서 도는 작은 모델)

```

# [부록 C] 『내 문서가 달라졌다』 6-4 - 저자 PC에서 2026-09-28 실행 확인 (Python
3.12, SQLite 3.49, CPU)
# 의미검색.py - 문단을 '뜻의 좌표'(임베딩)로 바꿔 두고, 질문과 뜻이 가까운 문서를
찾는다
# 준비(한 번만): pip install sentence-transformers
# 사용: python 의미검색.py 색인 "C:\공유폴더\영업"
# python 의미검색.py 찾기 배송이 늦어진 건
import sys, json, pathlib
import numpy as np
from sentence_transformers import SentenceTransformer

모델이름 = "paraphrase-multilingual-MiniLM-L12-v2" # 한국어 포함 다국어, 내
PC(CPU)에서 돈다
여기 = pathlib.Path(__file__).parent
좌표파일, 목록파일 = 여기 / "의미색인.npy", 여기 / "의미색인.json"
읽을확장자 = {".md", ".txt"}
_모델 = None

def 모델():
    global _모델
    if _모델 is None:
        _모델 = SentenceTransformer(모델이름, device="cpu") # 첫 실행 때 한 번
        내려받음
    return _모델

def 색인(폴더):
    조각들, 출처 = [], []
    for p in sorted(pathlib.Path(폴더).rglob("*")):
        if not p.is_file() or p.suffix.lower() not in 읽을확장자:
            continue
        본문 = p.read_text(encoding="utf-8", errors="ignore")
        제목 = 본문.strip().split("\n")[0].lstrip("# ")
        # 문단(빈 줄) 단위로 자른다 - 이 모델은 한 번에 짧은 글만 읽기 때문
        # 제목만 있는 줄은 빼고, 각 문단 앞에 문서 제목을 붙여 둔다
        for 문단 in [x.strip() for x in 본문.split("\n\n") if x.strip()]:
            if 문단.startswith("#"):
                continue
            조각들.append(제목 + " - " + 문단)
            출처.append(str(p))
    좌표 = 모델().encode(조각들, normalize_embeddings=True,

```

```

show_progress_bar=True)
np.save(작표파일, 작표.astype(np.float32))
json.dump({"출처": 출처, "조각": 조각들}, open(목록파일, "w", encoding="utf-8"), ensure_ascii=False)
print(f"색인 완료: 문서 {len(set(출처))}개, 문단 {len(조각들)}개")

def 찾기(질문, 몇개=5, 출력=True):
    작표 = np.load(작표파일)
    목록 = json.load(open(목록파일, encoding="utf-8"))
    q = 모델().encode([질문], normalize_embeddings=True)[0]
    점수 = 작표 @ q # 뜻이 얼마나 가까운가 (1에 가까울수록 비슷)
    문서점수 = {}
    for i, s in enumerate(점수): # 한 문서에서 가장 가까운 문단의 점수를 문서 점수로
        경로 = 목록["출처"][i]
        if s > 문서점수.get(경로, (-1, 0))[0]:
            문서점수[경로] = (float(s), i)
    순위 = sorted(문서점수.items(), key=lambda x: -x[1][0])
    if 출력:
        for 경로, (s, i) in 순위[:몇개]:
            print(f"[{s:.2f}] {pathlib.Path(경로).name}")
            print("..." + 목록["조각"][i][:60].replace("\n", " ") + "...")
    return [경로 for 경로, _ in 순위]

if __name__ == "__main__":
    if sys.argv[1] == "색인":
        색인(sys.argv[2])
    else:
        찾기(" ".join(sys.argv[2:]))

```

섞어찾기.py (6-5)

키워드 검색과 의미 검색 순위를 섞는다(RRF)

```

# [부록 C] 『내 문서가 달라졌다』 6-5 - 저자 PC에서 2026-09-28 실행 확인 (Python
3.12, SQLite 3.49, CPU)
# 섞어찾기.py - 키워드 검색과 의미 검색의 순위를 섞는다 (RRF: 순위 역수 합산)
# 먼저 두 색인을 만들어 둔다:
#     python 키워드검색.py 색인 "C:\공유폴더\영업"
#     python 의미검색.py 색인 "C:\공유폴더\영업"
# 사용: python 섞어찾기.py 낚기 지연
import sys, io, contextlib, pathlib
import 키워드검색, 의미검색

K = 60      # 순위 점수를 완만하게 만드는 상수. 원 논문(Cormack 외, 2009)이 쓴 값
위쪽 = 10   # 각 검색에서 위쪽 10개만 섞는다 (아래쪽 잡음이 끼어들지 않게)

def 섞어찾기(질문, 몇개=5, 출력=True):
    with contextlib.redirect_stdout(io.StringIO()):          # 각자 출력은 숨긴다
        키워드 = 키워드검색.찾기(질문.split(), 몇개=0)[:위쪽]
        의미 = 의미검색.찾기(질문, 출력=False)[:위쪽]
        점수, 출신 = {}, {}
        for 이름, 목록 in [("키워드", 키워드), ("의미", 의미)]:
            for 순위, 경로 in enumerate(목록, 1):
                점수[경로] = 점수.get(경로, 0) + 1 / (K + 순위) # 1등 1/61, 2등 1/62
        ...
        출신.setdefault(경로, []).append(f"{이름} {순위}등")
    결과 = sorted(점수, key=lambda p: -점수[p])
    if 출력:
        for 경로 in 결과[:몇개]:
            print(f"{pathlib.Path(경로).name} ← " + ", ".join(출신[경로]))
        if not 결과:
            print("두 방식 모두 못 찾음 - 6-5의 진단 4단계로")
    return 결과

if __name__ == "__main__":
    섞어찾기(" ".join(sys.argv[1:]))

```

채점_12문항.py (6-5)

정답을 아는 질문 12개로 세 방식을 채점한다

```

# [부록 C] 『내 문서가 달라졌다』 6-5 - 저자 PC에서 2026-09-28 실행 확인 (Python
3.12, SQLite 3.49, CPU)
# 채점.py - 정답을 아는 질문 12개로 세 방식(키워드/의미/섞기)을 채점한다
import sys, io, contextlib, pathlib, importlib
모듈명 = sys.argv[1] if len(sys.argv) > 1 else "의미검색"
sys.argv = ["x"]
import 키워드검색 as K

시험 = [ # (질문, 정답 파일)
    ("납기 지연", "회의록_2025-11-03.md"),
    ("포장 불량", "클레임_2025-09-12.md"),
    ("단가 인상", "통화메모_2025-10-02.md"),
    ("대금 미지급", "업무일지_2025-08-21.md"),
    ("야근 수당", "인사메모_2025-06-30.md"),
    ("재고 부족", "창고현황_2025-10-20.md"),
    ("출장 경비 정산", "출장보고_2025-07-15.md"),
    ("KX-2041", "부품목록_2025.md"),
    ("A상사 재계약", "계약메모_2025-12-01.md"),
    ("품질 검사 기준", "품질기준_v2.md"),
    ("3월 월간 보고", "월보고_2025-03.md"),
    ("신입 안내", "교육자료_신입.md"),
]

def 섞기(목록들, k=60):
    점수 = {}
    for 목록 in 목록들:
        for 순위, 경로 in enumerate(목록, 1):
            점수[경로] = 점수.get(경로, 0) + 1 / (k + 순위)
    return [p for p, _ in sorted(점수.items(), key=lambda x: -x[1])]

def 등수(목록, 정답):
    names = [pathlib.Path(p).name for p in 목록]
    return names.index(정답) + 1 if 정답 in names else None

M = importlib.import_module(모듈명)
import 섞어찾기 as H
행 = []
for 질문, 정답 in 시험:
    with contextlib.redirect_stdout(io.StringIO()):

```

```

kw = K.찾기(질문.split(), 몇개=0)
sem = M.찾기(질문, 출력=False)
hy = H.섞어찾기(질문, 출력=False)
행.append((질문, 등수(kw, 정답), 등수(sem, 정답), 등수(hy, 정답)))
def 표(x): return "-" if x is None else str(x)
print("질문 | 키워드 | 의미 | 섞기   (정답 문서 등수, -는 못 찾음)")
for q, a, b, c in 행:
    print(f"{q} | {표(a)} | {표(b)} | {표(c)}")
for 이름, j in [("키워드", 1), ("의미", 2), ("섞기", 3)]:
    top1 = sum(1 for r in 행 if r[j] == 1)
    top5 = sum(1 for r in 행 if r[j] is not None and r[j] <= 5)
    print(f"{이름}: 1등 {top1}/12, 5등 안 {top5}/12")

```

check.ps1 — 아침 점검 3문항 (9-1)

8-1의 아침 자동 실행 결과를 세 질문으로 점검합니다. 메모장에 붙여넣고 C:\ai\check.ps1로 저장합니다. PowerShell이 읽는 파일이라 한글이 들어가도 되지만, 저장할 때 인코딩을 '**UTF-8(BOM)**'으로 골라야 윈도우 기본 PowerShell에서 한글이 안 깨집니다(메모장 '다른 이름으로 저장' 창 아래쪽 인코딩 선택).

```

$dir = "C:\ai\morning"
$today = Get-Date -Format "yyyy-MM-dd"
$flag = Get-Item "$dir\start.flag" -ErrorAction SilentlyContinue # run.bat이
    실행 시작 때 만드는 표시(8-2)
$out = if ($flag) { Get-ChildItem $dir -Filter "$today*.md" | Where-Object
    { $_.LastWriteTime -gt $flag.LastWriteTime } | Select-Object -First 1 }
$src = (Get-ChildItem "$dir\inbox" -File).Count
$done = if ($out) { (Select-String -Path $out.FullName -Pattern '^~ ').Count }
    else { 0 }
$err = (Select-String -Path "$dir\run.log" -Pattern 'error|fail|실패' -Encoding
    utf8).Count
"1. 오늘 결과물: " + $(if ($out) { $out.Name } else { '없음 <- 실패' })
"2. 건수: 원본 $src / 결과 $done " + $(if ($src -ne $done) { '<- 불일치' } else
    { '' })
"3. 로그 속 에러 줄: $err " + $(if ($err -gt 0) { '<- 확인' } else { '' })

```

실행:

```
powershell -ExecutionPolicy Bypass -File C:\ai\check.ps1
```